

DelosDB MVCC Design

N2O Version Chain · Main+Delta Separation · Snapshot Isolation · Clean Architecture

DelosDB Engineering · June 2026 · Confidential Draft

Sources: PostgreSQL · MariaDB/InnoDB · H2 · LeanStore · Umbra · HyPer

Research: Wu et al. VLDB 2017 · Alhomssi & Leis VLDB 2023 · Pavlo CMU 2023

) and peer-reviewed research (Wu et al. VLDB 2017, Alhomssi & Leis VLDB 2023). The proposal covers the four Wu et al. design axes, the precise ar

1 The MVCC Design Space — Four Axes

Wu et al. (VLDB 2017) established the definitive framework for MVCC design decisions. Every MVCC system makes exactly four choices. Getting any one wrong creates either correctness gaps, scalability ceilings, or architectural debt that is very expensive to change later. DelosDB must make all four explicitly, not by default.

Axis	Question	Production choices
Concurrency Control	What protocol prevents anomalies?	2PL: Oracle, InnoDB, SAP HANA OCC: Hekaton, HyPer SSI: PostgreSQL, Serializable SQL Server
Version Storage	Where do old row versions live?	Append-Only: PostgreSQL, Hekaton Delta: Oracle, InnoDB, SQL Server Time-Travel: SAP HANA Adaptive: LeanStore (Alhomssi 2023)
Garbage Collection	When and how are dead versions removed?	Tuple-level: PostgreSQL (autovacuum), MySQL Transaction-level: Hekaton, NuoDB Cooperative: H2 (MVStore GC)
Index Management	What do index entries point to?	Physical: PostgreSQL (HOT chains), Hekaton Logical: InnoDB, Oracle, NuoDB, HyPer

2 Why PostgreSQL's Approach Is Wrong for DelosDB

PostgreSQL's MVCC is the most studied and most cautionary model. Its four specific problems are documented by Andy Pavlo (CMU, 2023) and confirmed in the Wu et al. VLDB 2017 survey. DelosDB must explicitly reject each one:

✗ Table bloat

Append-only storage keeps dead tuples in-place next to live tuples. A table receiving heavy updates grows unboundedly until autovacuum reclaims space. Autovacuum is a background process that cannot keep up under sustained write load, causing table scans to process dead tuples alongside live ones.

✗ Oldest-to-Newest (O2N) version chain

Index entries point to the OLDEST tuple version. A primary-key lookup must traverse the entire version chain from oldest to newest to find the current version. Under heavy write load, chains of 100+ versions force every point lookup to do $O(\text{chain_length})$ traversal just to find the current committed data.

✗ Full-row copy on every update

Even updating a single column copies the entire row to a new physical location. For wide rows (many columns), this produces $O(\text{columns} \times \text{updates})$ write amplification. InnoDB's delta approach writes only the changed columns to the undo log — for an 8-byte integer update in a 120-byte row, InnoDB writes ~16 bytes; PostgreSQL writes 120 bytes.

✗ Physical index pointers

PostgreSQL index entries contain the physical heap location (page, slot) of each tuple version. When a tuple is updated (creating a new physical location), every index that covers any updated column must be updated with the new location. This is HOT (Heap-Only Tuple) optimization — but it only helps when the new tuple fits in the same page, which is not guaranteed.

Pavlo (CMU, 2023): "if someone is going to build a new MVCC DBMS today, they should not do it the way PostgreSQL does. Its design is a relic of the 1980s and before the proliferation of log-structured system patterns from the 1990s. We did not find another DBMS doing MVCC the way PostgreSQL does it in our 2018 VLDB paper."

3

DelosDB MVCC Design Decisions

Concurrency Control	Multi-Version Snapshot Isolation (MV-SSI) No lock manager required. Readers never block writers; writers never block readers. Write-write conflicts detected at version-chain boundary (MvccWriteConflictException). MvccSnapshot captures active-transaction exclusion set at transaction open — already implemented and
Version Storage	N2O: Main Table + In-Memory Version Store (InnoDB model, Umbra execution) Current committed version always in MvccPageFile (like InnoDB's clustered index). Old versions in a separate MvccVersionStore (like InnoDB's undo log, but in-memory associated with pages via mapping table — Umbra's design). No table bloat — dead versions are never in the page file
Garbage Collection	Tuple-level background vacuum with watermark pinning MvccVacuumWatermark already designed and implemented. Background vacuum thread prunes MvccVersionStore entries older than the oldest active transaction's snapshot. Main page file (MvccPageFile) is never bloated — vacuum only touches MvccVersionStore.
Index Management	Logical pointers — index entries map to row key, not physical page location DelosMvccIndex already uses NavigableMap<Object, LinkedHashSet<K>> keyed by index value → row key (logical). Index entries survive physical row movement. Index never needs to be updated when a version chain entry is pruned. On update, index entry for new value added,

4

Target Architecture — Component Design

The target architecture separates MVCC into five independent components. Each has a single responsibility and a narrow interface. No component imports implementation classes from another — only interfaces. This is the architectural discipline the production systems that failed (PostgreSQL's t_xmin/t_xmax baked into the physical tuple) violated, and the discipline the successful ones (InnoDB's clean undo-log separation) honored.

Component	Responsibility	Current state	Gap
MvccTransactionRegistry	Transaction lifecycle: begin, commit, abort	MvccTransactionManager already correct design (A51)	No lock state/chain. Seq via published immutable snapshotOf() holds global lock during visibility checks.
MvccSnapshot & MvccCapturedVisibility	Point-in-time transaction visibility. Immutable snapshot of NewMvccSnapshotManager	Already correct design (A51)	Capturing snapshot still holds global lock.
MvccVersionStore	Per-row N2O chain of OLD committed versions	MvccVersionStore already correct design (A51) MvccTable.LinkedHashMap holds ALL versions (current + Full)	Main page file, not separated from history. Full can materializes all.

Component	Responsibility	Current state	Gap
MvccPageFile (Main Table Store)	Always holds the CURRENT committed PageBuffer. MvccTable is still the authoritative source.	PageBuffer MvccTable MvccPageFile MvccVersionStore.	Append-only page store exists. MvccTable is still the authoritative source.
MvccVacuumEngine	Background pruning of MvccVersionStore. MvccTable is still the authoritative source.	MvccVersionStore MvccTable MvccPageFile MvccVersionStore. MvccTable is still the authoritative source.	Background pruning of MvccVersionStore. MvccTable is still the authoritative source.

5

The Three Key Changes Required

5.1

Concurrency model — from global lock to versioned read path

The current design serializes every operation through one of three monitors: MvccTransactionManager, MvccTable, or MvccVersionChain. This means a scan and a concurrent write serialize completely — MVCC's fundamental promise is not yet implemented. The fix has two parts:

Part A: Lock-free transaction status publication

MvccCapturedVisibility (A51) is already the correct design — a frozen immutable snapshot of transaction states that visibility checks can use without touching the live MvccTransactionManager. The remaining gap is that MvccTransactionManager.snapshot() still holds the global lock while copying transaction state. The fix: publish an immutable TransactionSnapshot atomically on every commit/abort, readable without any lock by concurrent scan threads.

```
// Current: all reads and writes hold the same monitor
public synchronized MvccTransactionStatus statusOf(MvccTransactionId id) { ... }

// Target: volatile published snapshot — lock-free read path
private volatile TransactionSnapshot publishedSnapshot = TransactionSnapshot.EMPTY;

// On commit (write path — still synchronized):
public synchronized MvccCommitSequence commit(MvccTransaction tx) {
    state.status = COMMITTED; state.commitSequence = next++;
    publishedSnapshot = publishedSnapshot.withCommitted(tx.id(), state.commitSequence);
    return state.commitSequence;
}

// On read (lock-free — any thread):
public MvccTransactionStatus statusOf(MvccTransactionId id) {
    return publishedSnapshot.statusOf(id); // no lock, immutable read
}
```

Part B: Row-granularity locking — ReentrantReadWriteLock per row

MvccVersionChain currently holds synchronized(this) — one lock per row chain. This is the right granularity (row-level), but the wrong lock type. synchronized pins virtual threads (JEP 444, JDK 21). Replace with ReentrantReadWriteLock: multiple concurrent readers on the same row chain, exclusive lock only for mutations (update/delete). MvccTable's own lock guards the LinkedHashMap structure (insert/lookup), not the chain contents. This follows the MapDB ByteBufferVol.growLock pattern (Apache 2.0, verified from source).

```
public final class MvccVersionChain {
    // Replace: synchronized methods on `this`
    // With:    ReadWriteLock — concurrent reads, exclusive writes
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();

    public Optional visibleValue(MvccCapturedVisibility visibility) {
        lock.readLock().lock(); // multiple readers can proceed concurrently
        try {
            for (MvccVersion v : newestFirst) {
                if (visibility.isVisible(v)) return Optional.ofNullable(v.value());
            }
        } finally {
            lock.readLock().unlock();
        }
    }
}
```

```

        }
        throwIfPrunedHistoryWouldHaveBeenVisible(visibility);
        return Optional.empty();
    } finally { lock.readLock().unlock(); }
}

public void update(V newValue, MvccTransaction tx, MvccCapturedVisibility visibility) {
    lock.writeLock().lock();    // exclusive write – blocks other writers on this row
    try { ... } finally { lock.writeLock().unlock(); }
}
}

```

5.2 Version storage — Main Table + MvccVersionStore separation

This is the central architectural change. Today MvccTable holds ALL versions (current and historical) in one LinkedHashMap. The target separates them:

	Current design	Target design
Current version	In MvccVersionChain.newestFirst[0] (head of the list)	In MvccPageFile — the physical page store. Non-MVCC read: O(1) page lookup, no chain.
Old versions	Also in MvccVersionChain.newestFirst[1..n]	In MvccVersionStore — in-memory N2O chain associated with pages via MvccRowDirectory.
MVCC read	Traverse newestFirst list from head until visible version found	1. Read current from MvccPageFile. 2. If visible → done. 3. Else traverse MvccVersionStore chain.
Table scan	synchronized openScan() materializes entire visible result set before return	Lazy cursor: iterate MvccPageFile pages; per row check current → MvccVersionStore.
Vacuum	Prunes MvccVersionChain inside MvccTable.rows LinkedHashMap	Prunes MvccVersionStore only. MvccPageFile never holds dead versions.
Durability	PageBackedMvccTable appends all versions to page file	MvccPageFile: current version. MvccVersionStore: in-memory, rebuilt on open from MvccPageMutationLog (already exists).

The MvccVersionStore interface is narrow — exactly five methods:

```

public interface MvccVersionStore {
    /** Push old version when current is about to be replaced. */
    void pushOldVersion(K key, MvccVersion oldVersion);

    /** Traverse old versions for MVCC visibility (N2O order). */
    Optional findVisible(K key, MvccCapturedVisibility visibility);

    /** Prune versions safe to remove under the given watermark. */
    MvccCleanupResult prune(K key, MvccCommitSequence oldestRetained);

    /** Prune all entries for all keys below watermark (vacuum pass). */
    MvccCleanupResult vacuumAll(MvccCommitSequence oldestRetained);

    /** True if this key has any old versions that still need retention. */
    boolean hasRetainedVersions(K key, MvccCommitSequence oldestRetained);
}

```

5.3 Lazy scan cursor — fix the scan materialization blocker

MvccTable.openScan() currently materializes the entire visible result set into a List before returning. This holds the table lock for the entire duration of visibility evaluation. For a 100K-row table, this blocks every concurrent writer for the duration of the scan. The fix is a lazy cursor that advances one row at a time:

```
public final class MvccLazyScan implements AutoCloseable {
    private final Iterator keyIterator; // snapshot of KEY SET at scan-open time
    private final MvccPageFile mainTable;
    private final MvccVersionStore versionStore;
    private final MvccCapturedVisibility visibility;
    private MvccRow current;

    // Key insight: snapshot key set once (fast), read values lazily (no lock held)
    MvccLazyScan(Set keysAtOpenTime, MvccPageFile main,
        MvccVersionStore store, MvccCapturedVisibility vis) {
        this.keyIterator = List.copyOf(keysAtOpenTime).iterator(); // key snapshot only
        this.mainTable = main;
        this.versionStore = store;
        this.visibility = vis;
    }

    public boolean next() {
        while (keyIterator.hasNext()) {
            K key = keyIterator.next();
            // 1. Check current version (no lock, 0(1))
            Optional currentVersion = mainTable.readCurrent(key);
            if (currentVersion.isPresent() && visibility.isCurrentVisible(key)) {
                current = new MvccRow<>(key, currentVersion.get()); return true;
            }
            // 2. Only if current not visible: check version store (rare MVCC read)
            Optional oldVersion = versionStore.findVisible(key, visibility);
            if (oldVersion.isPresent()) {
                current = new MvccRow<>(key, oldVersion.get()); return true;
            }
            // 3. Row not visible to this snapshot – skip, advance to next key
        }
        return false;
    }
}
```

6

Transaction Lifecycle — Data Safety Contract

Data safety means: committed data is never lost, uncommitted data is never visible to other transactions, and aborted data is cleaned up correctly. The durable structures already in DelosDB support this — the gap is that the in-memory transaction manager and version chain do not integrate with them consistently.

6.1

The four-phase write path

Phase	Operation	Durable write	Visibility
1. Begin	Allocate MvccTransactionId. Capture snapshot (active set + visibleThrough).	None — transaction not yet durable.	None. Versions written with this txId are ACTIVE
2. Write	INSERT: push new version to MvccPageFile. (marked uncommitted in tuple header). UPDATE: push old version to MvccVersionStore, then write new version to MvccPageFile. DELETE: mark current in MvccPageFile as deleted-by this txId.	MvccPageMutationLog.append for every page write. MvccVersionStore.write carries: txId, key, value, operation type.	Visible only to owning tx (snapshot.owner check in MvccCapturedVisibility)
3. Commit	Assign commit sequence (monotonic counter). Publish to volatile TransactionSnapshot. fsync MvccTransactionOutcomeLog COMMITTED entry.	MvccTransactionOutcomeLog.write before returning to caller. fsync MvccTransactionOutcomeLog COMMITTED entry.	Immediately visible to all new snapshots taken after this commit.

Phase	Operation	Durable write	Visibility
4. Abort	Mark all versions written by this txId as aborted in MvccVersionStore. MvccPageFile: revert current version to previous from MvccVersionStore.	MvccTransactionOutcomeLog (fsync — prevents resurrection after crash during recovery).	Never Visible (x) Aborted versions pruned by vacuum

6.2 Recovery and restart safety

After restart, the in-memory MvccTransactionManager is empty. MvccVersionStore is empty. The only durable truth is MvccPageFile, MvccPageMutationLog, and MvccTransactionOutcomeLog. Recovery must reconstruct the in-memory state from these three before accepting any new transactions:

Recovery sequence on database open:

1. Read MvccTransactionOutcomeLog
 - Identify all COMMITTED transactions and their commit sequences
 - Identify all ABORTED transactions
 - Any transaction in MvccPageMutationLog but NOT in outcome log = UNRESOLVED
2. Apply MvccPageMutationLog to MvccPageFile
 - For each log entry belonging to a COMMITTED transaction:
write committed version to MvccPageFile (current version)
 - For each log entry belonging to an ABORTED transaction:
discard the version (it was never committed)
 - For UNRESOLVED transactions:
throw MvccUnresolvedTransactionOutcomeException (already exists)
or apply configurable resolution policy (abort-on-unresolved is safest)
3. Rebuild MvccVersionStore
 - From all remaining committed versions in MvccPageFile,
reconstruct the N20 chain of old versions per row
 - Only versions newer than the oldest retained watermark need reconstruction
4. Rebuild MvccTransactionRegistry
 - From MvccTransactionOutcomeLog: populate committed + aborted transactions
 - Publish initial TransactionSnapshot
 - Begin accepting new transactions

Key: this is already the right structure – MvccPageRecoveryRunner exists.

Gap: the integration with MvccTransactionManager rehydration is missing.

7

Vacuum — Garbage Collection Design

PostgreSQL's autovacuum is the canonical example of GC done wrong at the architectural level: it must reclaim dead tuples from the same physical pages as live data, causing table bloat and requiring a background process that can fall behind under write load. DelosDB's main+delta separation eliminates this problem at the root: dead versions are in MvccVersionStore only, not in MvccPageFile. Vacuum never touches the main page file.

7.1 What vacuum does and does not do

Vacuum operation	Touches	Does NOT touch
Prune old versions	MvccVersionStore chains older than oldest watermark	MvccPageFile — never. Live pages are not bloated.
Prune index candidates	MvccIndexStore: remove candidates whose row version no longer exists	Live index entries for current rows
Prune aborted versions	MvccVersionStore: immediately on abort, not deferred	Committed version chains

Vacuum operation	Touches	Does NOT touch
Prune transaction catalog	MvccTransactionManager: remove states older than GC horizon	Recent transactions needed by active snapshots

7.2 Vacuum watermark and snapshot lease — already designed

MvccVacuumWatermark + MvccSnapshotLease (A44–A45) are already implemented and address exactly the problem that Alhomssi & Leis VLDB 2023 identified in existing systems: a single long-running OLAP query can cause transactional throughput to collapse because vacuum cannot proceed past the oldest active snapshot. DelosDB's explicit MvccSnapshotLease makes this pinning intentional and observable:

```
// Open a retained snapshot – explicitly pins the vacuum watermark
// (not implicit pinning from a long-running query that nobody knows about)
try (MvccSnapshotLease lease = txManager.openSnapshot(transaction)) {
    MvccSnapshot snapshot = lease.snapshot();
    // Long-running analytical scan – vacuum cannot proceed past lease.watermark()
    // but all other transactions can still commit and vacuum can still prune
    // versions newer than this watermark's minimum
    scanAllRows(snapshot);
} // lease.close() – watermark released, vacuum can now proceed

// Background vacuum sees:
MvccCommitSequence safeWatermark = txManager.oldestRetainedVisibleThrough();
versionStore.vacuumAll(safeWatermark); // only prunes what ALL holders have seen
```

8 Index Management — Logical Pointers

The index design follows the logical-pointer model used by InnoDB, Oracle, NuoDB, and HyPer — the four systems Wu et al. VLDB 2017 identified as having better index management than PostgreSQL's physical pointers.

8.1 Why logical pointers are correct

- Index entry maps index value → row key (logical identifier). Physical page location is never stored in the index.
- When a version chain entry is pruned by vacuum, the index is not touched. Only MvccVersionStore loses the entry.
- When a row is updated, the index for the NEW value is added; the old value's candidate set is marked stale. Physical pages can be reorganized without updating any index.
- Visibility recheck at scan time: `index.lookup(indexValue)` returns a set of candidate row keys; for each candidate, `visibleValue(snapshot)` is called. This is the PostgreSQL-guided rule already in `DelosMvccIndex.candidatesByKey`.

8.2 The current DelosMvccIndex is already correct

DelosMvccIndex uses `NavigableMap<Object, LinkedHashSet<K>>` — index value to set of candidate row keys. The lookup already rechecks visibility against the version chain before returning results. The only gap is that index update on writes must be integrated with the Main+Delta write path:

```
// On UPDATE of row with key K, old indexed value V_old, new value V_new:
// 1. Push old version to MvccVersionStore (before overwriting current)
versionStore.pushOldVersion(key, currentVersion);

// 2. Write new version to MvccPageFile (becomes new current)
mainTable.writeCurrent(key, newVersion, txId);
```

```
// 3. Update indexes (logical – just candidate sets, no physical locations)
for (DelosMvccIndex index : indexes) {
    Object oldIndexKey = index.extractKey(oldValue);
    Object newIndexKey = index.extractKey(newValue);
    if (!oldIndexKey.equals(newIndexKey)) {
        index.removeCandidate(oldIndexKey, key); // old value no longer maps here
        index.addCandidate(newIndexKey, key);    // new value maps to this row key
    }
    // If index key unchanged: no index update needed (key still maps to same row)
}
```

9

Future Lane — Adaptive Version Storage

Alhomssi & Leis VLDB 2023 propose an adaptive version storage scheme that automatically switches per-tuple between full-copy and delta-only versions based on observed update frequency. This is the Phase 2 optimization on top of the base N2O Main+Delta architecture proposed here:

Format	When used	Storage	Read path	Write path
Full copy (current baseline)	All tuples initially	Complete row value per version	Direct access — no reconstruction	Copy entire row to MvccVersionStore
Delta (hot tuples)	Tuples with high update frequency	Changed columns only (like InnoDB undo delta)	Reconstruct by applying deltas from current backward	Write only changed columns, lower write amplification
Adaptive switch	MvccVersionChain.deadVersionEstimate() exceeds threshold	Chooses format per tuple based on history	Transparent to caller	Transparent to caller

Prerequisite for adaptive storage: the MvccVersionStore must first be separated from MvccTable (Change 5.2 above). Adaptive storage is a version format decision inside MvccVersionStore — it cannot be designed until the store has its own interface. MvccVersionChain.deadVersionEstimate() already collects the hot/cold classification data needed to trigger the switch.

10

Implementation Sequence

#	Item	Changes required	JDK	Weeks	Status
M-1	MvccVersionChain: synchronized	Replace synchronized methods. Use capturedVisibility (not live catalog) for all visibility checks.	21	<1	NOW
M-2	MvccTransactionManager: volatile	Add volatile transactionSnapshot field. Publish on every commit/abort. statusOf() / commitSequenceOf() become lock-free.	21	1	NOW
M-3	MvccVersionStore interface + InMemoryVersionStore implementation	Define MvccVersionStore interface. InMemoryVersionStore backed by ConcurrentHashMap<K, List<MvccVersion<V>>>. Replace MvccVersionChain inside MvccTable.	21	1–2	NEXT
M-4	MvccTable: use MvccPageFile as MvccTable version store	MvccTable.update pushes old to MvccVersionStore then writes new to MvccPageFile. MvccTable no longer owns current versions.	21	1–2	NEXT
M-5	MvccLazyScan cursor (replace openScan)	Replace MvccLazyScan with key snapshot. Per-step: check MvccPageFile → MvccVersionStore. Remove synchronized openScan() from MvccTable.	21	1	NEXT

#	Item	Changes required	JDK	Weeks	Status
M-6	Recovery: MvccTransactionManager	Generate baseline from read Mvcc log. Rebuild committed/aborted state. Publish initial TransactionSnapshot. Apply MvccPageMutationLog for uncommitted.	20	1-2	NEXT
M-7	Index update integration with Main	DelosMvccIndex.addCandidate / removeCandidate called from DelosMvccTable write path (not from version chain).	21	1-5	NEXT
M-8	MvccVacuumEngine: vacuum MvccVersionStore	vacuum MvccVersionStore to call versionStore.vacuumAll(watermark). Keep MvccVacuumWatermark and MvccSnapshotLease (already correct).	21	0.5	NEXT
F-1	Adaptive version storage (full-copy)	MvccVersionStore format selector. Delta codec for changed-column encoding. Switch trigger: deadVersionEstimate() threshold.	21	4-6	FUTURE
F-2	Graveyard Index	Side structure for tombstones. Integrated with MvccVersionStore pruning.	21	4-6	FUTURE
F-3	Scalable commit protocol	Autonomous commit (Alhomssi & Leis VLDB 2023) Eliminates write-set touch on commit.	21B 2023	5	FUTURE

R

References

- [1] Wu, Y., Arulraj, J., Lin, J., Xian, R. & Pavlo, A. (2017). An Empirical Evaluation of In-Memory Multi-Version Concurrency Control. PVLDB 10(7), 781–792. The definitive survey establishing the four MVCC design axes.
- [2] Alhomssi, A. & Leis, V. (2023). Scalable and Robust Snapshot Isolation for High-Performance Storage Engines. PVLDB 16(6), 1426–1438. <https://www.vldb.org/pvldb/vol16/p1426-alhomssi.pdf> Source: Graveyard Index, adaptive version storage, scalable commit protocol.
- [3] Pavlo, A. (2023). The Part of PostgreSQL We Hate the Most. CMU Blog. <https://www.cs.cmu.edu/~pavlo/blog/2023/04/the-part-of-postgresql-we-hate-the-most.html> Source: O2N/N2O analysis, append-only problems, full-row copy amplification.
- [4] Leis, V. (2024). LeanStore: A High-Performance Storage Engine for NVMe SSDs. PVLDB 17(12), 4536–4545. Source: version chain architecture, adaptive storage model.
- [5] Freitag, M. et al. (2022). Memory-Optimized Multi-Version Concurrency Control for Disk-Based Database Systems. PVLDB. Umbra MVCC: in-memory version chains associated with pages via local mapping tables.
- [6] PostgreSQL source: `src/backend/access/heap/heapam.c`, `src/include/access/htup_details.h` (`t_xmin/t_xmax HeapTupleHeaderData`). Confirms O2N append-only storage with physical-pointer index entries. <https://github.com/postgres/postgres>
- [7] MariaDB/InnoDB source: `storage/innobase/row/row0vers.cc` (version visibility), `storage/innobase/trx/trx0purge.cc` (purge/vacuum), `storage/innobase/include/trx0roll.h` (undo log roll pointer). Confirms N2O delta storage, logical pointer indexes. <https://github.com/MariaDB/server>
- [8] CMU 15-445 Lecture 19: Multi-Version Concurrency Control. <https://15445.courses.cs.cmu.edu/spring2025/notes/19-multiversioning.pdf> Source: system-by-system design decision matrix.
- [9] Apache Derby / DelosDB: `delosdb-storage-mvcc` (`MvccVersionChain`, `MvccSnapshot`, `MvccTransactionManager`, `MvccTable`, `MvccTupleHeader`, `MvccVacuum`, `MvccPageMutationLog`, `MvccTransactionOutcomeLog`, `MvccRowDirectory`). License: Apache 2.0. <https://github.com/ggeorg/delosdb>
- [10] MapDB source: `org.mapdb.store.legacy.Volume.ByteBufferVol.growLock` (`ReentrantLock`). Source: `ReentrantReadWriteLock` pattern for concurrent readers + exclusive writers. License: Apache 2.0. <https://github.com/jankotek/mapdb>